

Pràctica 7.1 Arduino

Objectius de l'activitat

La present activitat té com a objectius fonamentals desenvolupar els següents muntatges;

- 1. Programar un LED**
- 2. Implementar un LED controlat per interruptor**
- 3. Simular un sistema d'engegada d'un sistema de il·luminació en funció del nivell de llum mesurat per un sensor LDR**
- 4. Controlar un ServoMotor**
- 5. ServoMotor controlat per potenciòmetre**

Es dedicaran 8 hores a l'aula a la realització de la pràctica. La memòria de la pràctica es lliurarà en format electrònic (.doc, .docx o .odt, pdf no). La pràctica s'avaluarà en base a les respostes proporcionades a les diverses qüestions que planteja el present enunciat i la presentació i netedat de la memòria.

Introducció al muntatge elèctric

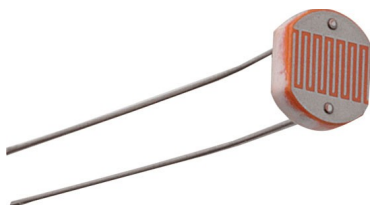
Material necessari:

1. Placa Arduino UNO
2. Sensor LDR
3. Placa de prototipus
4. Resistències
5. Cables d'interconnexió
6. ServoMotor
7. Potenciòmetre
8. Multímetre digital

Introducció:

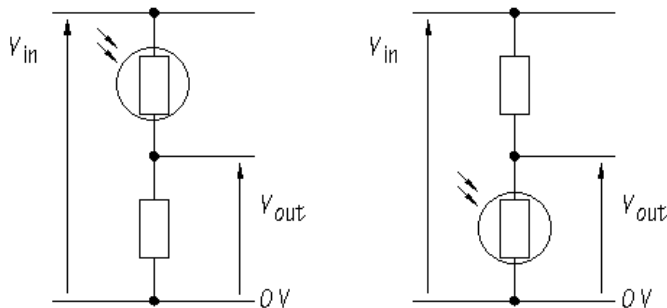
Què és un LDR?

Un LDR és una resistència variable, que varia el seu valor depenent de la quantitat de llum que incideix sobre la seva superfície. Quanta més intensitat de llum incideix en la superfície de la LDR menor serà la seva resistència i com menys llum incideix major serà la resistència. Solen ser utilitzats com a sensors de llum ambiental o com una fotocèl·lula que activa un determinat procés en absència o presència de llum.



Un **sensor de llum** es compon d'una LDR com a part d'un divisor de tensió resistiu.

Exemple:



$$V_{out} = ((R_{bottom} / (R_{bottom} + R_{top})) * V_{in})$$

Si la LDR és usada com **R_{top}** (resistència de dalt), com en el primer circuit, dóna tensió alta (HIGH) a la sortida quan la LDR està en la llum, i una tensió baixa (LOW) a la sortida quan la LDR està a l'ombra.

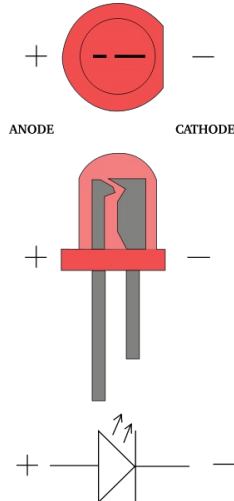
L'acció del divisor de tensió és inversa quan la LDR és usada com **R_{bottom}** (resistència de baix), enlloc de **R_{top}**, com en el segon circuit. El circuit dóna tensió Baixa (LOW) a la sortida quan la LDR està en la llum, i una tensió alta (HIGH) a la sortida quan la LDR està a l'ombra.

El circuit divisor de tensió donarà una tensió de la sortida que canvia amb la il·luminació, de forma inversament proporcional a la quantitat de llum que rebí (sensor de fosc).

Què és un díode LED?

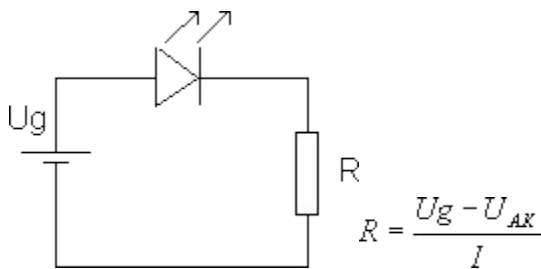
Un díode LED, acrònim anglès de *Light Emitting Diode* (díode emissor de llum) és un dispositiu semiconductor que emet llum incoherent d'espectre reduït quan se'n polaritza de forma directa la unió PN i és travessat per corrent elèctric. El color depèn del material semiconductor emprat en la construcció del díode, i pot variar des de l'ultraviolat, passant per l'espectre de llum visible, fins a l'infraroig.

El led té dos terminals clarament diferenciats, ànode (positiu) i càtode (negatiu), que podem diferenciar fàcilment per la seva diferent longitud i per una asimetria en la forma del dispositiu, tal i com podem observar a la figura següent:



L'emissió de llum es produeix quan el díode és en estat de conducció. En estat de conducció, la caiguda de tensió entre ànode i càtode és $U_{AK} = 1,5 \text{ V}$. La intensitat lluminosa que genera depèn del corrent que el travessa, habitualment funcionen amb intensitats compreses entre 10 i 30 mA.

El circuit d'aplicació típic és el següent:



El resistor té la funció de limitar la intensitat del corrent (normalment entre 10 i 30 mA) que travessi el LED, i adequar-la a la lluminositat requerida.

El valor d' R serà en Ω , si les tensions són en V i la intensitat en A.

Vegem un exemple: Un LED que ha de funcionar a partir d'una pila de 9 V, necessita un resistor que limiti el corrent del circuit a 15 mA, aleshores el seu valor es calcula:

$$R = \frac{U_g - U_{AK}}{I} = \frac{9V - 1,5V}{0,015A} = \frac{7,5V}{0,015A} = 500\Omega$$

El valor de resistència de $500\ \Omega$ no coincideix amb cap valor normalitzat, els valors normalitzats més propers són $470\ \Omega$ i $510\ \Omega$. Per muntar el circuit real, haurem d'escollir entre un d'aquests valors, en concret, el més proper, que és el de $510\ \Omega$.

Nosaltres utilitzarem un díode LED per a simular el nostre sistema de il·luminació automàtic: s'encendrà només quan el nostre sensor de llum detecti fosc.

La placa Arduino, a la seva sortida digital número 13, ja incorpora una resistència de polarització pensada per a connectar-hi un LED, per la qual cosa no en posarem cap resistència per fer aquesta funció i connectarem el LED entre el pin digital 13 i GND (terra). Hauríem de fer-ho si el connectéssim a qualsevol altra de les sortides digitals per tal de no fer malbé el LED.

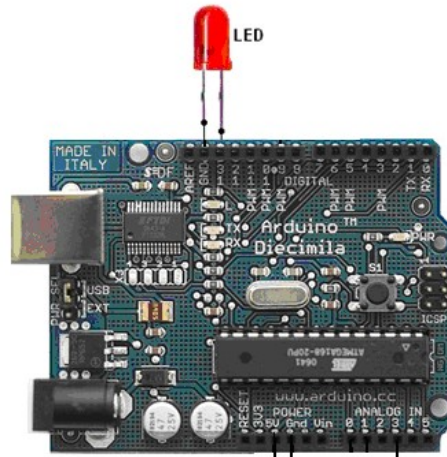
El nostre sensor de llum l'implementarem amb la LDR com a R_{bottom} , seguint l'esquema elèctric de la pàgina següent.

Per fer tot el connexionat elèctric utilitzarem una placa de prototipus o *protoboard*, seguint les instruccions del professorat.

Podeu consultar en qualsevol moment la pàgina:

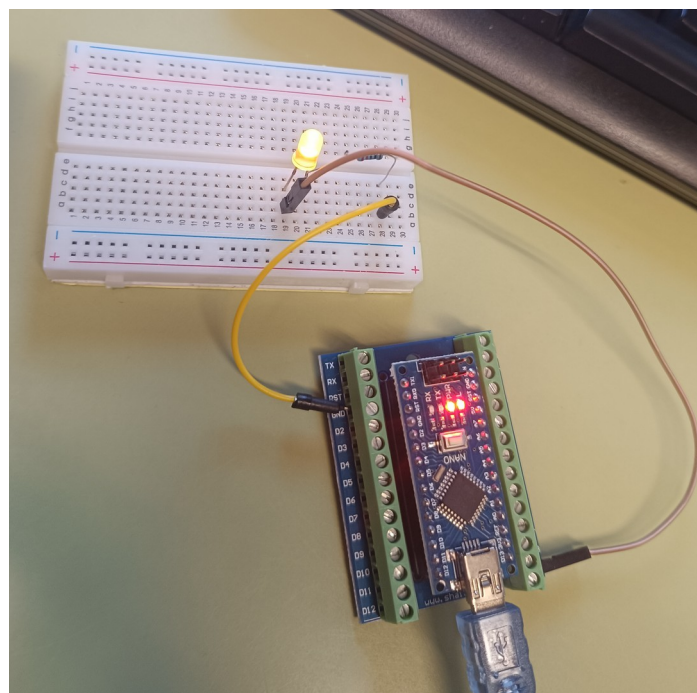
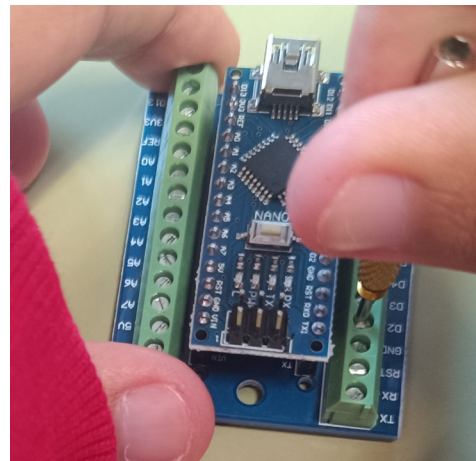
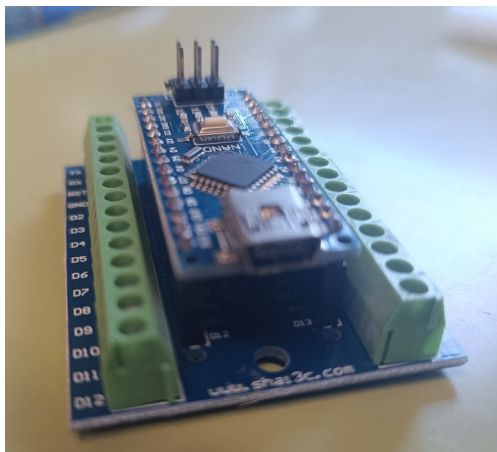
<http://arduino.star.com/documentacion/general/protoboard-que-es-y-como-funcional>

Activitat 1: Programar un LED intermitent



Esquema elèctric a implementar:

TASCA 1: reproduïu el muntatge indicat a la figura anterior amb l'ajuda de la placa de prototipus i els components de que disposeu. Feu una fotografia del muntatge a on es vegi clarament la interconnexió entre els diversos components i la placa i enganxeu-la aquí.



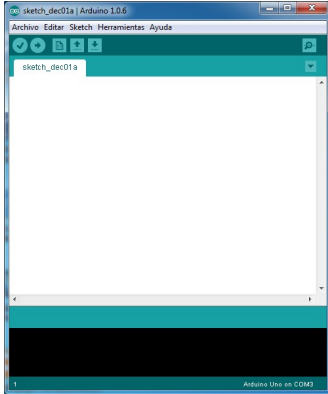
PART 1.2: Programant Arduino.

Material necessari:

1. LED color vermell
2. Placa d'Arduino
3. Cable de connexió Arduino-USB
4. Software de programació Arduino

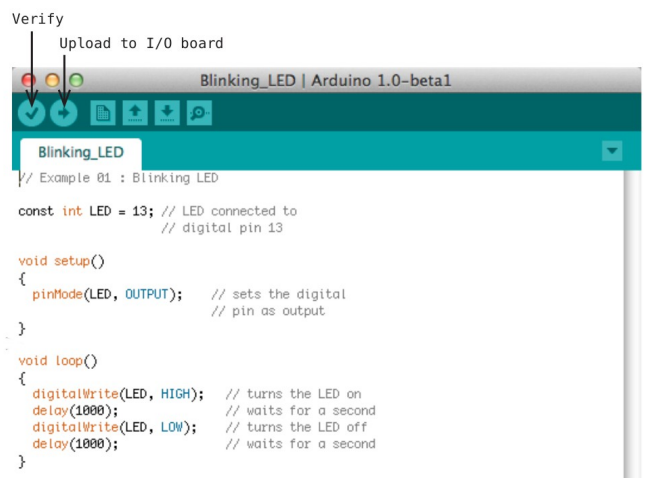
Introducció:

Obrim l'entorn de programació Arduino, ja instal·lat al nostre ordinador, i ens trobarem amb la pantalla següent:



En base doncs a l'esquema elèctric implementat i al que acabem d'explicar, el codi que necessitem per a programar la placa Arduino per a implementar el nostre sistema de LED parpellejant seria el següent (pots ometre els comentaris després del marcador //).

```
// Example 01 : LED intermitent
const int LED = 13; // LED connectat al
// pin digital 13
void setup()
{
    pinMode(LED, OUTPUT);
    // configura el pin digital com a
    sortida
}
void loop()
{
    digitalWrite(LED, HIGH); // engega el
    LED
    delay(1000); // espera 1 segon (1000
    milisegons)
    digitalWrite(LED, LOW); // apaga el
    LED
    delay(1000); // espera 1 segon
}
```



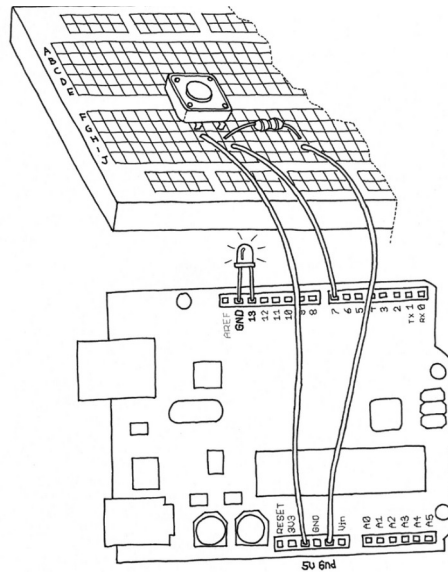
TASCA 2: enganxeu el codi anterior en la interfície de programació Arduino, carregueu el programa a la placa amb el botó de la fletxa (carregar) i comproveu que el sistema implementat funciona correctament.

Qüestió 1.

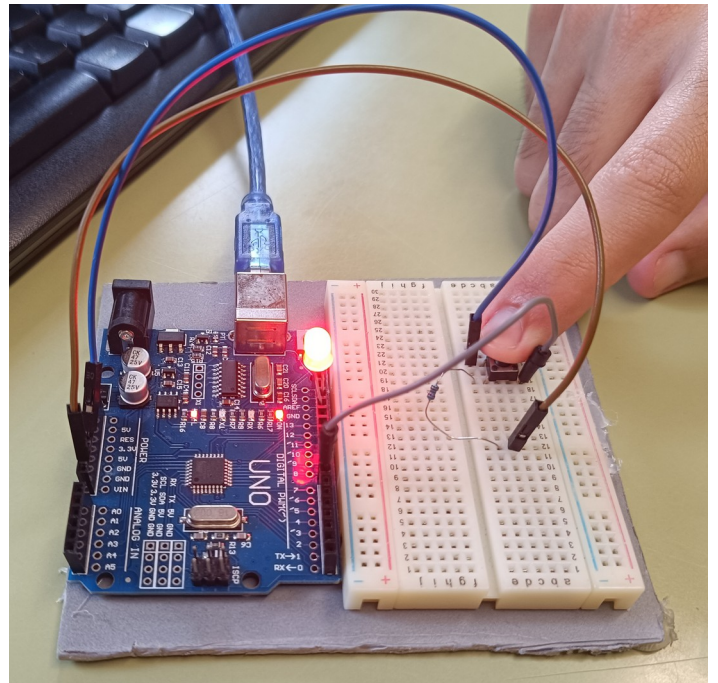
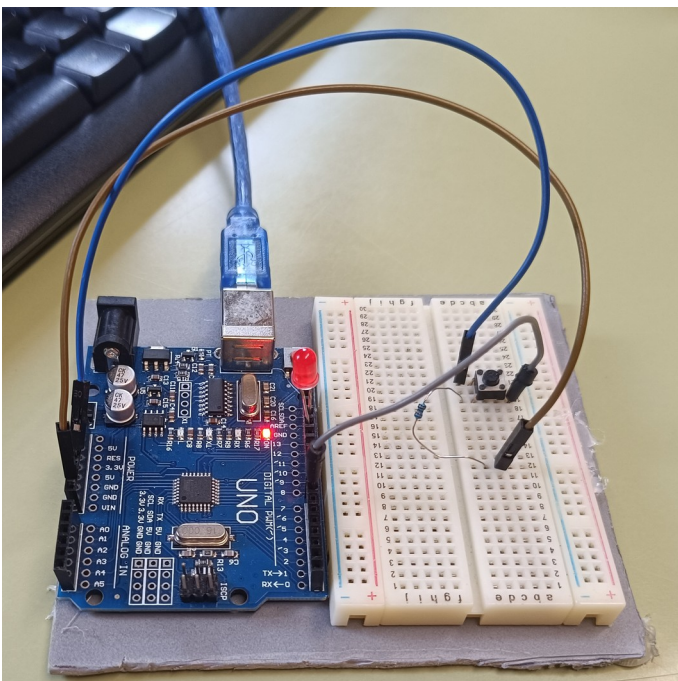
Canvia el valor de la funció `delay(1000)` per `delay(500)` i `delay(5000)`. Canvia-la primer en el primer lloc on apareix (després de `digitalWrite(LED,HIGH);` i després darrere de la línia `digitalWrite(LED,LOW);` i després en tots dos llocs junts i alternativament. Què és el que ocorre? Perquè?

Depenent del número que li fiques va més ràpid o més lent. És a dir, canvia el temps en que s'encén i s'apaga la bombeta.

Activitat 2. Implementar un LED controlat per interruptor



TASCA 3: reproduïu el muntatge indicat a la figura anterior amb l'ajuda de la placa de prototipus i els components de que disposeu. Feu una fotografia del muntatge a on es vegi clarament la interconnexió entre els diversos components i la placa i enganxeu-la aquí.



PART 3.2: Programant Arduino.

Material necessari:

1. Placa Arduino
2. LED vermell
3. Cable de connexió Arduino-USB
4. Software de programació Arduino
5. Resistència de 10KΩ
6. Interruptor tipus switch

Introducció:

Obrim l'entorn de programació Arduino, ja instal·lat al nostre ordinador. En base doncs a l'esquema elèctric implementat i al que acabem d'explicar, el codi que necessitarem per a programar la placa Arduino per a implementar el nostre sistema de LED controlat per un interruptor és el següent (pots ometre els comentaris després del marcador //).

```
// Example 02: Engega el LED si es prem l'interruptor
const int LED = 13;
// número de pin per a conenctar el LED
const int BUTTON = 7; // el pin on es connecta el interruptor
int val = 0;
// val és la variable on emmagatzemem l'estat del interruptor

void setup() { // funció per a inicialitzar els paràmetres
    pinMode(LED, OUTPUT);
    // Declara el pin 13 com a sortida
    pinMode(BUTTON, INPUT); // declara el pin 7 com a entrada
}

void loop(){ // bucle que es repeteix una vegada inicialitzat
    val = digitalRead(BUTTON); // llegim el pin connectat a
    // l'interruptor per a veure si està pres. (Voltatge HIGH)
    // comprovem si el butó està pres
    if (val == HIGH) { // en cas de que sí
        digitalWrite(LED, HIGH); // engega el LED
    } else {
        digitalWrite(LED, LOW); // apaga el LED
    }
}
```

TASCA 4: enganxeu el codi anterior en la interfície de programació Arduino, carregueu el programa a la placa amb el botó de la fletxa (carregar) i comproveu que el LED s'engega en prémer el interruptor

Qüestió 2.

Canvia la línia on posa

```
if (val == HIGH) {
```

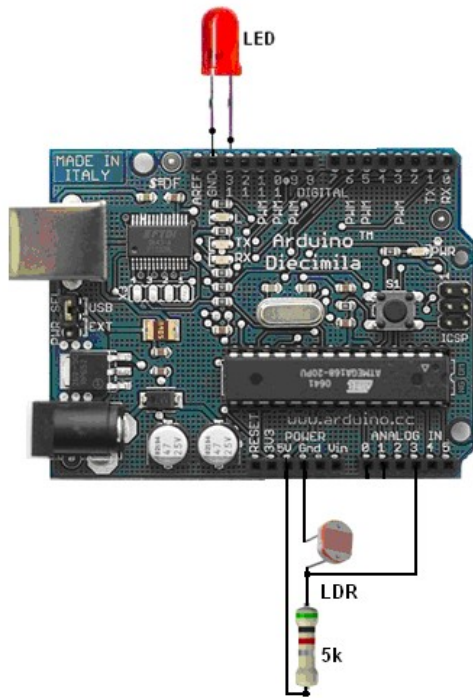
per el següent valor:

```
if (val == LOW) {
```

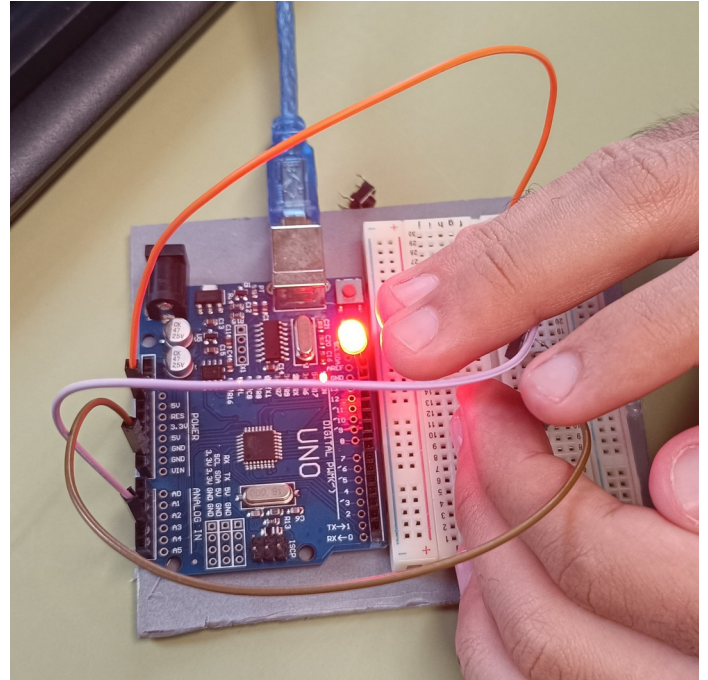
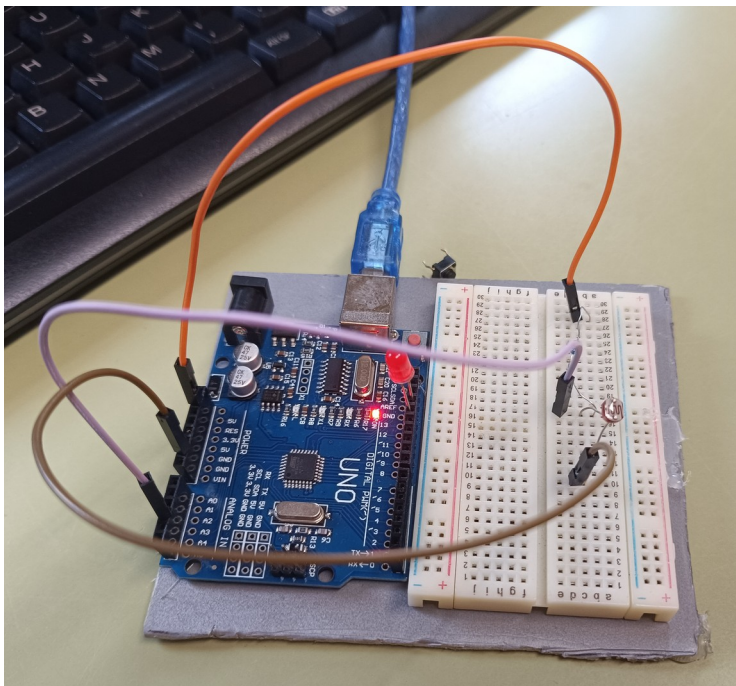
Què és el que ocorre? Perquè?

Quan li dones al botó, s'apaga i, quan no l'estàs apretant, la bombeta està encesa.

Activitat 3. LED controlat per fotoresistor



TASCA 5: reproduïu el muntatge indicat a la figura anterior amb l'ajuda de la placa de prototipus i els components de que disposeu. Feu una fotografia del muntatge a on es vegi clarament la interconnexió entre els diversos components i la placa i enganxeu-la aquí.



Material necessari:

1. LED color vermell
2. Placa d'Arduino
3. Cable de connexió Arduino-USB
4. Software de programació Arduino
5. Resistència 5K Ω
6. Resistència LDR

En ella escriurem el nostre sketch que programarà la placa Arduino per encendre el LED quan detecti que el nivell de llum està per sota d'un determinat llindar.

Com hem llegit abans, quan la LDR és usada com R_{bottom} (resistència de baix), el sensor de llum dóna tensió Baixa (LOW) a la sortida quan la LDR està en la llum, i una tensió alta (HIGH) a la sortida quan la LDR està a l'ombra. Com que les entrades analògiques donen un valor entre 0 i 1024, determinarem que no hi ha llum quan el sensor ens lliuri un valor superior a 512.

En base doncs a l'esquema elèctric implementat i al que acabem d'explicar, el codi que necessitarem per a programar la placa Arduino per a implementar el nostre sistema d'engegada automàtica del LED en funció del valor de llum ambiental seria el següent:

```
int LightPin = 3; // selecciona el pin d'entrada per
// al sensor de llum
int ledPin = 13; // selecciona el pin per al LED
int val = 0; // variable per a emmagatzemar el valor
//capturat pel sensor de llum
int llindar=512; // variable per a emmagatzemar el valor
// de llindar de llum detectada a partir del qual
// s'encendrà el LED

void setup() {
    pinMode(ledPin, OUTPUT); // declara el ledPin en mode //sortida
}
void loop() {
    val = analogRead(LightPin); //llegim el valor del
    // sensor
    if(val>llindar)
    {
        digitalWrite(ledPin, HIGH); // si el valor
        // mesurat està per sobre del llindar, encen el LED
    }
    else
    {
        digitalWrite(ledPin, LOW); // si el valor mesurat
        // està per sota del llindar, apaga el LED
    }
}
```

TASCA 6: enganxeu el codi anterior en la interfície de programació Arduino, carregueu el programa a la placa amb el botó de la fletxa (carregar) i comproveu tapant i destapant el sensor de llum que el sistema implementat funciona correctament.

Qüestió 3.

Prova a canviar el valor de la variable llindar en valors entre 0 i 1023.

Així on posa: `int llindar=512;`

pots posar: `int llindar=280;`

o `int llindar=750;`

Què és el que ocorre? Perquè?

Quan el valor és baix necessites més llum per a encendre'l.

Activitat 4. Servomotor amb Arduino

Aquest exemple fa ús de la llibreria "Servo.h" per a Arduino. Pots consultar:

<http://arduino.cc/en/Tutorial/sweep>

<http://www.electroensaimada.com/servomotor.html>

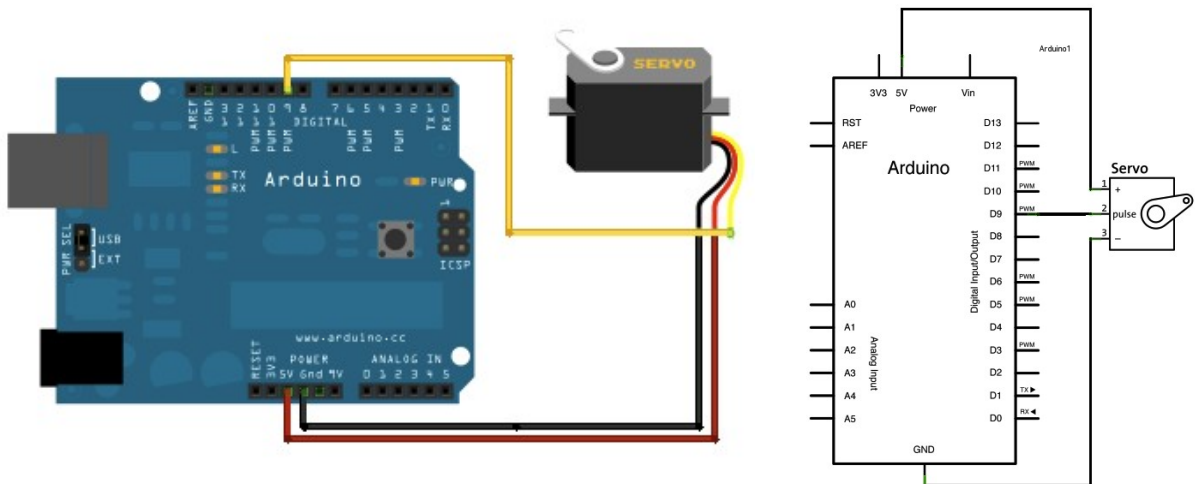
Hardware necessari

1. Placa Arduino
2. Servomotor
3. Cables de circuit
4. Cable de connexió Arduino-USB
5. Software de programació Arduino

Circuit

Els servomotors tenen tres cables: Alimentació, terra i senyal. El cable d'alimentació típicament és vermell i deu estar connectat al pin de 5V en la placa d'Arduino. La terra és típicament de color negre o marró i deu connectar-se al pin de terra de l'Arduino. El pin del senyal és típicament groc, taronja o blanc i deu connectar-se al pin 9 en la placa Arduino.

Bàsicament un servomotor és un motor de corrent continu amb un potenciòmetre que li permet saber la posició en la que es troba i així poder controlar-la. Per a controlar el servomotor se li envien polsos cada 20ms, es a dir 50 Hz. L'amplària del pols és el que modifica l'angle de gir, el que es coneix com PWM o codificació per ample de pols. Aquesta amplària varia segons el servomotor però normalment va entre 0,5 i 2,5 ms, encara que pot variar.



Codi font : (no cal posar els comentaris al codi -el text després de // -)

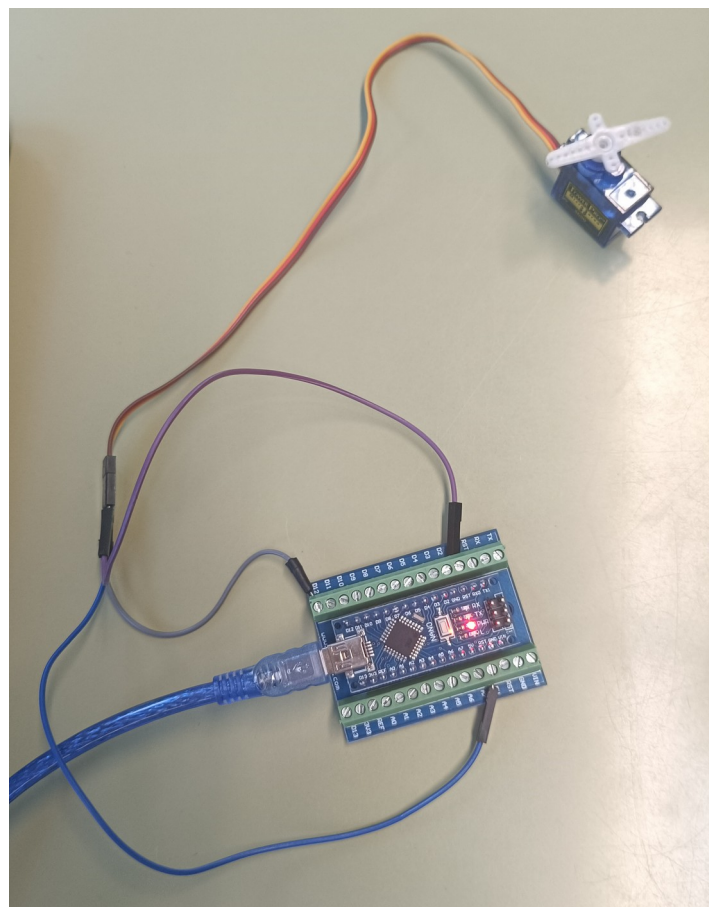
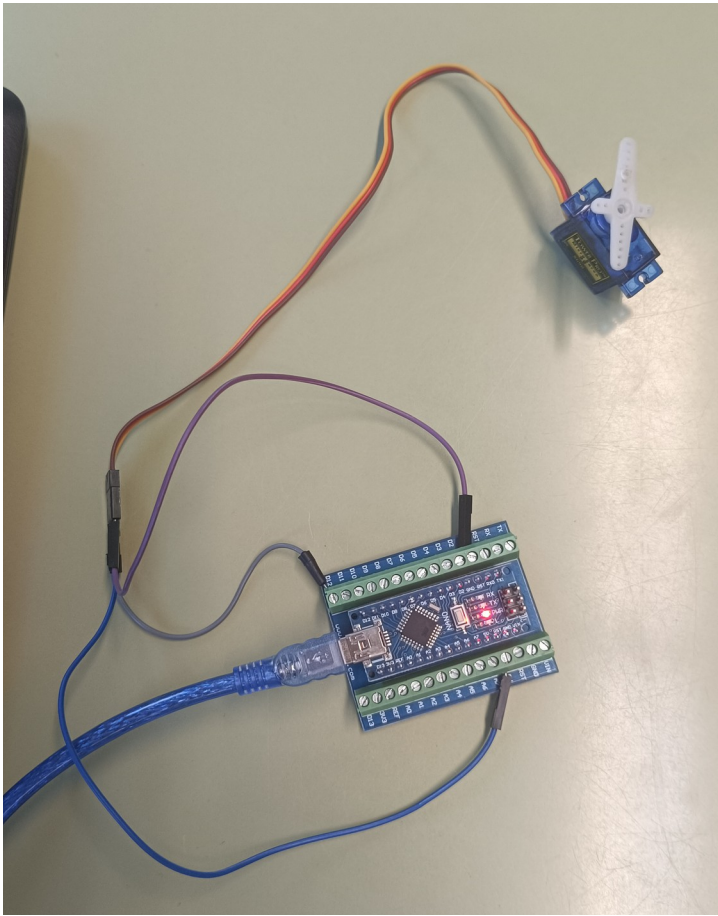
```
// Exemple de domini públic. Servomotor amb potenciòmetre
#include <Servo.h> //incorpora les llibreries de funcions de servo
Servo myservo; // crea un objecte Servo

int pos = 0; // variable que emmagatzema la posició
// del servomotor

void setup() //inicialitzem els paràmetres
{
  myservo.attach(9); // adjuntem el pin 9 al control del servo
}

void loop() //bucle que es repeteix sempre una vegada inicialitzat
{
  for(pos = 0; pos < 180; pos += 1)//Va de 0 a 180 graus(variable pos)
  {
    // en passos d'un grau
    myservo.write(pos); // comunica al servo la posició
  }
  // amb la variable pos
  delay(15); // Espera 15 ms per a continuar
  for(pos = 180; pos>=1; pos-=1) //Va de 180 a 0 graus decremantant
  {
    myservo.write(pos); // comunica al servo la posició
  }
  //amb la variable 'pos'
  delay(15); // espera 15 ms
}
```

TASCA 7: enganxeu el codi anterior en la interfície de programació Arduino, carregueu el programa a la placa amb el botó de la fletxa (carregar) i comproveu que el servo motor va canviant l'angle successivament



Qüestió 4.

Prova a posar en la primera sentència for el següent (canvi 180 per 90)

for (pos = 0; pos < 90; pos += 1)

en la segona

for (pos = 90; pos >= 1; pos -= 1)

Què és el que ocorre i perquè?

**En la primera en ves d'anar de 0 a 180 graus, va de 0 a 90
i en la segona, va de 90 a 1.**

Activitat 5. ServoMotor controlat per potenciòmetre

Aquest exemple fa ús de la llibreria “Servo.h” per a Arduino. Pots consultar aquest exemple en:

<http://arduino.cc/en/Tutorial/Knob>

Hardware necessari

1. Placa Arduino
2. Servomotor
3. Cables de circuit
4. Potenciòmetre
5. Cable de connexió Arduino-USB
6. Software de programació Arduino

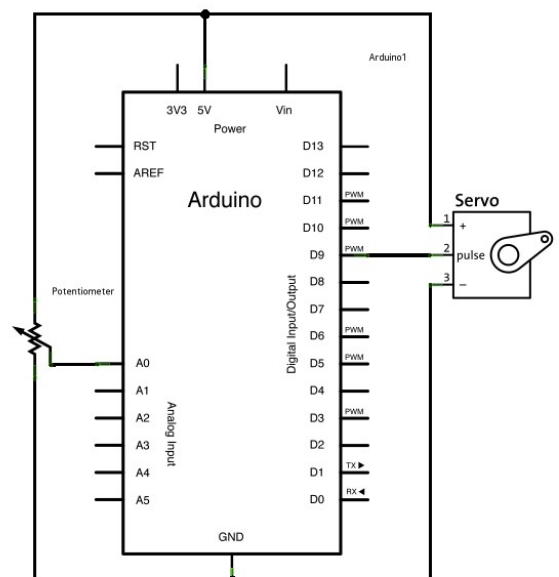
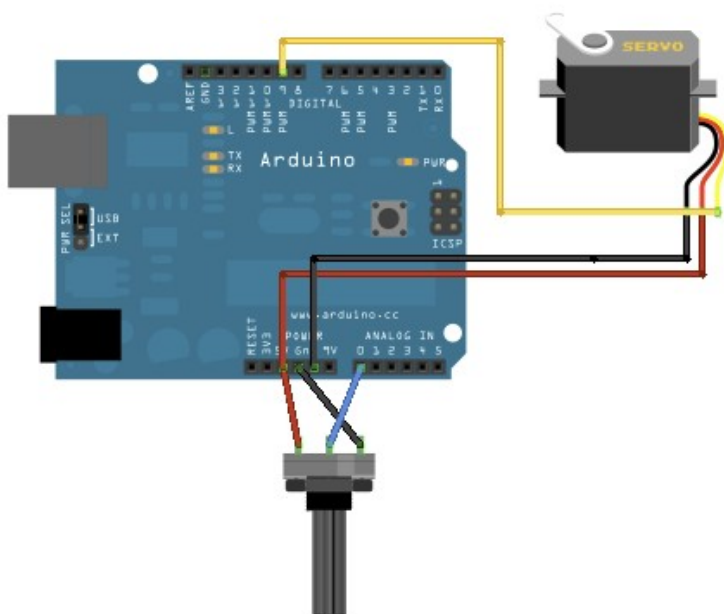
Un **potenciòmetre** és una resistència variable on podem girar des de la mínima al màxim valor de resistència. Actua com un divisor de tensió on el valor del pin de lectura pren un valor de voltatge corresponent a la posició del gir.

El potenciòmetre deu estar cablejat de forma que els dos pins externs estiguen connectats a l'alimentació (+5V) i l'altre al terra.

El pin del mig deu estar connectat a una entrada analògica de l'Arduino (A0).

Convertirem el valor de voltatge de A0 a un temps de nivell actiu de la sortida PWM del pin 9 (p.ex. 20 ms = 90°), així comunicarem l'angle que deu girar i amb la llibreria de <Servo.h> obtindrem aquest valor a partir de la lectura del potenciòmetre amb la funció `val = map(val, 0, 1023, 0, 179);`

On obtenim un valor entre 0 i 179 proporcional al valor de la variable val (lectura analògica del pin A0) que va des de 0 a 5 Volts amb 10 bits de precisió (de 0 a 1023 valors).



Codi font (pots ometre els comentaris després del marcador //)

```
// Controlar un servo motor amb un potenciòmetre
#include <Servo.h> // incorporem la llibreria de funcions de servomotor

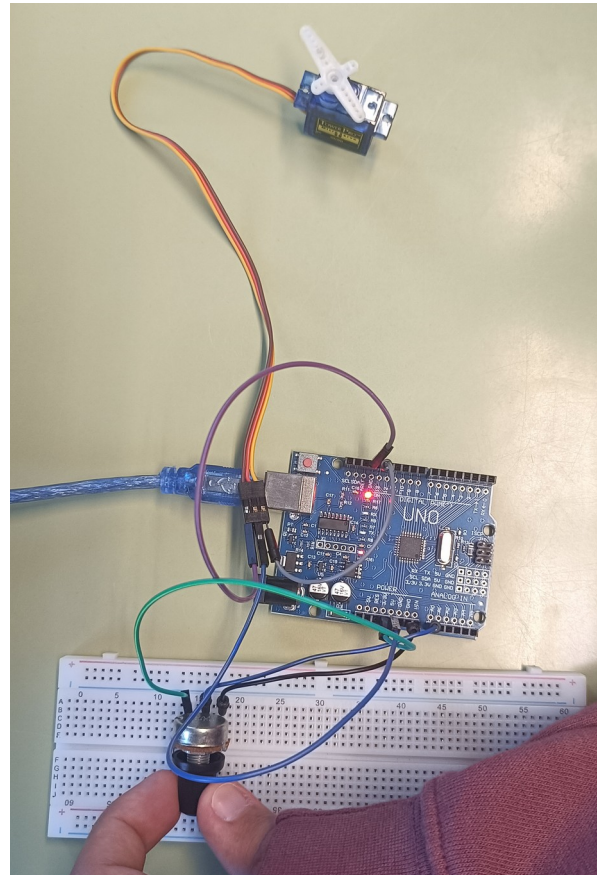
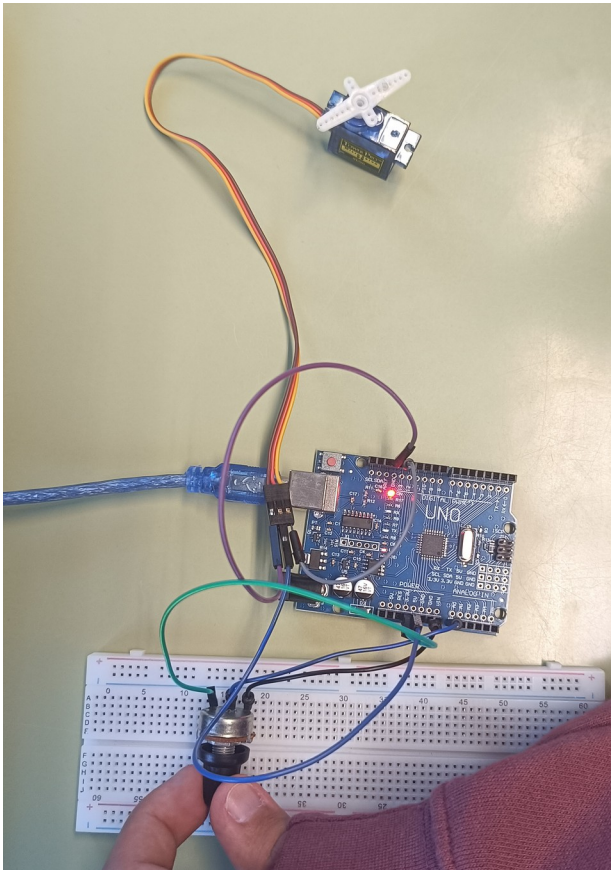
Servo myservo; // creem un objecte Servo per a manipular el motor

int potpin = A0; // entrada analògica per llegir el potenciòmetre
int val; // variable per a emmagatzemar el valor de la entrada analògica

void setup() // inicialització del muntatge
{
    myservo.attach(9); // A l'objecte Servo li adjudiquem la entrada
    // de control al pin 9 del Arduino
}

void loop() // bucle a repetir
{
    val = analogRead(potpin); // llig el senyal del potenciòmetre
    // entre 0 i 1023
    val = map(val, 0, 1023, 0, 179); // Obtenim el valor corresponent //proporcional a la
    // entrada A0
    myservo.write(val); //Establim la posició del servo
    delay(15); // Esperem 15 ms
}
```

TASCA 8: enganxeu el codi anterior en la interfície de programació Arduino, carregueu el programa a la placa amb el botó de la fletxa (carregar) i comproveu que el servo motor va canviant l'angle proporcionalment al gir del potenciòmetre



Qüestió 5.

Prova a posar en la sentència el següent: (canviar 179 per 89):
Què és el que ocorre i perquè?

```
val = map(val, 0, 1023, 0, 179);  
val = map(val, 0, 1023, 0, 89);
```

Se mou menys distància quan gires el potenciòmetre.